Blinkit Personal Chef

"Cook at Home" — Recipe-to-Cart Intelligence

Product Requirements Document
Version 1.0 • Proposed Concept • Quick-Commerce Prototype

Document Type	Product Requirements Document
Project Type	Product case study / prototype
Platform	Blinkit-inspired Quick Commerce Application
Primary Feature	Recipe-to-Cart / Cook at Home
Prepared By	Melody · IT Engineering, L.D. College of Engineering (GTU)
Status	Proposed Concept

Table of Contents

1. Executive Summary
2. Product Vision
3. Problem Statement
4. Product Hypothesis
5. Proposed Solution & Core User Journey
6. Target Users & Personas
7. User Stories
8. Functional Requirements
9. UX Flow (Screen-by-Screen)
10. Technical Architecture
11. Data Model
12. API Design
13. MVP Scope & Phased Roadmap
14. Success Metrics & KPIs
15. Risks & Mitigations
16. Competitive Positioning
17. Appendix — Core Innovation Pipeline

1. Executive Summary

Blinkit Personal Chef is a proposed "Cook at Home" feature for a Blinkit-style quick-commerce application. Instead of requiring users to search individually for every grocery item needed to prepare a dish, the product converts a single meal intent — e.g. "Paneer Butter Masala" — into a complete, servings-aware, inventory-aware grocery basket in a few taps.

The core innovation is not recipe generation itself, but the end-to-end pipeline that connects **dish intent** → **recipe** → **serving-scaled ingredient list** → **normalized ingredients** → **SKU matching** → **pack optimization** → **pantry removal** → **one-tap cart** → **post-delivery cooking guidance.** This reframes Blinkit from a product-search platform into a meal-intent commerce platform, with the potential to materially increase basket size and product discovery.

Positioning statement: ***"You choose the dish. Blinkit handles the groceries."***

2. Product Vision

Transform grocery shopping from:

“I know what groceries I need, so let me search for them.”

into:

“I know what I want to eat. Figure out everything I need.”

The Cook at Home experience lets a customer search for a dish, select the number of people they are cooking for, and automatically receive a complete recipe, required ingredient quantities, matching Blinkit products, an estimated total cost, the ability to remove ingredients already at home, alternatives for unavailable items, a one-tap "Add All to Cart" action, and step-by-step cooking instructions.

3. Problem Statement

Today, a user who decides to cook a specific dish must independently: search for a recipe online, understand it, derive an ingredient list and quantities, then open Blinkit and search for each ingredient one by one (paneer, tomatoes, butter, cream, spices…), select the right products, check what they already own, add every item individually, and keep referring back to the recipe while cooking.

The user's actual intent — *"I want to cook Paneer Butter Masala tonight"* — is translated by today's UX into 10–15 separate product searches. This produces significant decision friction and search friction that a meal-intent-aware system can eliminate.

4. Product Hypothesis

If Blinkit understands the user's meal intent rather than requiring individual product searches, it can automatically convert **Dish** → **Recipe** → **Ingredients** → **Inventory** → **Cart**.

Hypothesis: If users can convert a dish into a complete, customizable grocery cart within a few taps, they will experience less shopping friction while increasing basket size, product discovery, conversion, and repeat

usage.

5. Proposed Solution & Core User Journey

When a user searches a supported dish (e.g. "Paneer Butter Masala"), the search results surface a **Cook at Home** card alongside normal product results, inviting the user to start the guided flow.

Core User Journey

```
Search Dish
↓
Cook at Home
↓
Select Servings
↓
Generate Recipe
↓
Calculate Ingredients
↓
Match Ingredients with Available Products
↓
Review Ingredient Cart
↓
Remove Ingredients Already at Home
↓
Choose Substitutions if Required
↓
Add All to Cart
↓
Checkout
↓
Delivery
↓
Guided Recipe Mode
```

Critical Distinction: Required Quantity vs. Purchasable Quantity

The system must never overstate precision. If a recipe requires 75 g of butter but the smallest available pack is 100 g, the UI must clearly separate **"75 g required"** from **"Recommended: Amul Butter · 100 g pack."** This distinction is essential for product credibility and is treated as a first-class requirement throughout the design (FR-04, FR-06, Section 64).

Pack Optimization

The system selects the purchasable combination of packs that best satisfies the recipe requirement while minimizing cost, waste, and mismatch with preference. Conceptually:

```
Best SKU = minimum( cost + waste_penalty + preference_penalty )
subject to: purchased_quantity >= required_quantity
```

Pantry Removal & Smart Pantry

Users can deselect ingredients they already own before checkout. A later "My Pantry" feature lets users maintain a standing list of staples (salt, oil, turmeric, cumin, etc.) that are automatically pre-excluded from future recipe baskets, with a lightweight review prompt to avoid stale data.

One-Tap Cart & Recipe Mode

All remaining ingredients are added to the cart in a single batch action. After delivery, the same session becomes a lightweight, step-by-step cooking assistant (with optional timers and progress tracking) rather than ending at checkout.

6. Target Users & Personas

Persona	Core Problem	Core Need
Student / Young Professional	Knows what to eat but not always the ingredients, quantities, or steps.	"Just tell me what I need and send everything."
Beginner Home Cook	Follows YouTube/Instagram recipes and constantly switches apps.	One integrated cooking workflow.
Busy Family	Needs to cook for 3–6 people; requirements scale with family size.	Automatic serving calculations.
Experimental Cook	Enjoys new dishes but unfamiliar recipes need unfamiliar ingredients.	Automatically generated ingredient baskets.

7. User Stories

ID	As a customer, I want to…
US-01	Search for a dish and purchase everything needed to prepare it.
US-02	See a "Cook at Home" option when searching for supported dishes.
US-03	Specify how many people I am cooking for.
US-04	Have ingredient quantities automatically adjusted to serving size.
US-05	Have every recipe ingredient mapped to an available grocery product.
US-06	Remove products I already have at home.
US-07	Change the recommended brand/product for any ingredient.
US-08	Get alternatives when a recommended product is unavailable.
US-09	Add all required ingredients to my cart simultaneously.
US-10	Access step-by-step cooking instructions after purchasing ingredients.

8. Functional Requirements

FR-01 — Dish Search Recognition

Distinguish product intent (e.g. "paneer", "tomato") from dish intent (e.g. "paneer butter masala", "dal makhani"). When dish-intent confidence is high, surface the Cook at Home card.

FR-02 — Recipe Retrieval

Each supported recipe is stored as structured data, e.g.:

```
{
"recipe_id": "PBM001",
"name": "Paneer Butter Masala",
"base_servings": 4,
"prep_time": 15,
"cook_time": 30,
"difficulty": "Easy",
"diet": "Vegetarian"
}
```

FR-03 — Structured Ingredients

Ingredients are structured, not free text, e.g.:

```
{
"ingredient_id": "ING_PANEER",
"name": "Paneer",
"quantity": 400,
"unit": "g",
"required": true
}
```

FR-04 — Serving Scaling

Base formula:

```
New Quantity = Base Quantity × (Requested Servings / Base Servings)
Example: 400 g paneer (base 4) → 200 g for 2 people
```

Spices, salt, and oil should support ingredient-specific scaling rules rather than strict linear scaling, since they don't always scale proportionally with servings.

FR-05 — Ingredient Normalization

Recipe language variants ("Paneer", "Indian cottage cheese", "Fresh paneer") must resolve to a single canonical ingredient ID (e.g. **PANEER**) so catalog matching is reliable.

FR-06 — Ingredient-to-SKU Mapping

```
Recipe Ingredient → Ingredient Normalization → Product Candidate Retrieval →
Inventory Filter → Pack Optimization → Ranking → Recommended SKU
```

FR-07 — Product Replacement

Every ingredient supports a "Change Product" action so the algorithm never removes user choice.

FR-08 — Ingredient Removal

Each ingredient can be added/removed, with the running total price updating immediately.

FR-09 — Out-of-Stock Handling

When a recommended product is unavailable, the system surfaces a valid substitute and clearly discloses that a substitution occurred.

FR-10 — Recipe Feasibility

A recipe is marked "Currently unavailable to cook" if any **critical ingredient** (one whose absence materially prevents the dish, e.g. paneer for Paneer Butter Masala) cannot be sourced. Optional garnish-type ingredients do not block feasibility.

FR-11 — Batch Add to Cart

```
POST /cart/add-recipe
{
"recipe_id": "PBM001",
"servings": 4,
"products": [
{ "sku_id": "SKU123", "quantity": 2 },
{ "sku_id": "SKU456", "quantity": 1 }
]
}
```

Response includes: successfully added products, failed products, price changes, inventory changes, and suggested replacements.

9. UX Flow (Screen-by-Screen)

Screen 1 — Search

User searches “Paneer Butter Masala”.

Screen 2 — Search Results

A “Cook at Home” card appears above normal product results, summarizing time, difficulty, and diet type.

Screen 3 — Serving Selector

User picks number of people (2 / 3 / 4 / 6). Copy reassures: right quantities, no missing ingredients, remove what you have, easy substitutions.

Screen 4 — Ingredient Review

Full ingredient list with “Required” vs “Buying” quantities shown per item (e.g. Paneer — Required 400 g / Buying 2 × 200 g).

Screen 5 — Pantry Customization

Checklist of common staples (salt, oil, garam masala, etc.) the user can mark as already owned, with a live savings total.

Screen 6 — Cart CTA

Single summary card: item count, % of required ingredients covered, total price, and “Add N Items to Cart”.

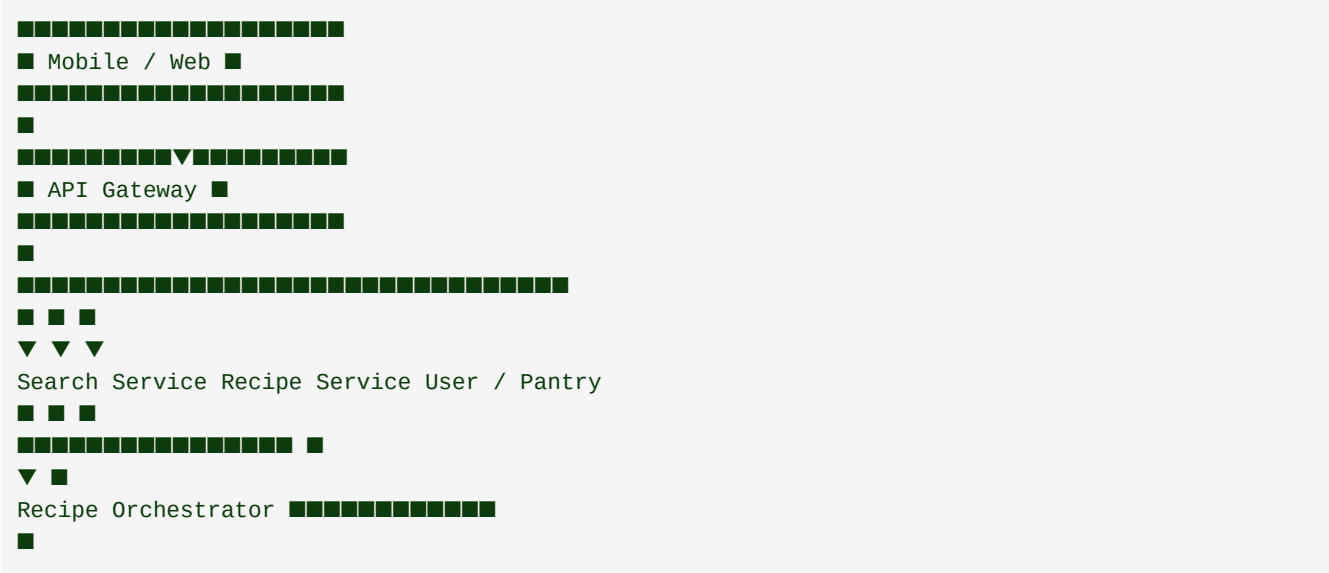
Screen 7 — Checkout

Feeds into the existing Blinkit payment/address/delivery/promotions flow — not rebuilt.

Screen 8 — Cooking Mode

Post-delivery, step-by-step guided cooking with prep/cook time and step navigation.

10. Technical Architecture



Suggested Prototype Stack

Layer	Recommendation
Frontend	React + Vite, Tailwind CSS, React Router, Axios, Lucide React
Backend	Node.js + Express.js — auth, recipe APIs, ingredient calculation, matching, cart, pantry, AI integration
Database	MongoDB — users, recipes, ingredients, products, inventory, pantry, carts, orders, recipe_sessions
AI Layer	Gemini / OpenAI API for dish-intent understanding, substitutions, and personalization (not for raw quantities)

Recommended Hybrid AI Architecture

- **Layer 1 — Verified Recipes:** curated database of ~50–100 popular dishes for MVP (e.g. Paneer Butter Masala, Dal Makhani, Chole, Pav Bhaji, Veg Biryani, Rajma, Palak Paneer, Shahi Paneer, Poha, Upma).
- **Layer 2 — Deterministic Serving Engine:** quantity scaling is computed in code, not by an LLM.
- **Layer 3 — AI:** used for dish-intent understanding, query normalization, recommendations, substitutions, and natural-language explanation — never as the sole source of quantities.

Design principle: Verified Recipe Database + Deterministic Quantity Engine + AI Understanding/Personalization — never "user request → LLM → blindly trust everything."

11. Data Model

Recipe

```
Recipe
-------
_id, name, slug, description, image, baseServings,
prepTime, cookTime, difficulty, dietType, cuisine,
ingredients[], steps[], tags[]
```

Ingredient

```
{
"canonicalName": "Paneer",
"aliases": ["Indian Cottage Cheese", "Fresh Paneer"],
"category": "Dairy",
"defaultUnit": "g"
}
```

Product

```
Product
-------
_id, name, brand, ingredientId, packSize, unit,
price, inventory, image, category
```

Pantry

```
Pantry
------
userId, ingredients[ ingredientId, quantity, unit, updatedAt ]
```

Recipe Session

```
RecipeSession
-------------
userId, recipeId, servings, ingredients,
removedIngredients, selectedSKUs, substitutions,
estimatedTotal, createdAt
```

12. API Design

```
GET /api/search?q=paneer+butter+masala
GET /api/recipes/:recipeId
POST /api/recipes/:recipeId/generate { "servings": 4 }
POST /api/recipes/:recipeId/match-products
PATCH /api/recipe-session/:id/ingredients
POST /api/cart/add-recipe
```

Search Intelligence

The search layer should tolerate spelling variation ("panner butter masala", "paneer buter masala", "PBM") via fuzzy matching, aliases, embeddings/vector search, or LLM classification, all resolving to the same canonical recipe.

Dietary Preferences & Allergy Safety

MVP filters should prioritize Vegetarian, Jain, No Onion/Garlic, and spice level, given the Indian market context. Allergen handling (dairy, nuts, gluten, soy) must be treated carefully: substitutions should be validated, not casually generated, and AI recommendations must never be presented as medical guidance.

13. MVP Scope & Phased Roadmap

MVP (Must Have)

- Dish search & Cook at Home card
- 20–50 curated recipes
- Serving selector
- Ingredient quantity calculation
- Ingredient-to-product mapping
- Pack/product recommendation
- Remove ingredient / replace product
- Price calculation
- One-click add-all-to-cart
- Recipe instructions
- Out-of-stock handling

Phase 2

- User pantry ("My Pantry")
- Dietary preferences & Jain recipes
- Smart substitutions
- Budget filter
- Recipe recommendations, favorites, recently cooked
- Nutrition information

Phase 3 — AI Personal Chef

A conversational interface that can recommend a dish from a loose request, e.g.:

```
User: I want something spicy using paneer.
Chef: How many people?
User: 4
Chef: I recommend Paneer Tikka Masala. ~35 minutes; missing ingredients cost ■328.
Generate basket?
```

Phase 4 — Multimodal Pantry

Users photograph their kitchen/fridge; AI detects likely available ingredients and automatically narrows the shopping list — a strong future extension, explicitly out of scope for MVP.

Explicit Non-Goals for MVP

- Automatic refrigerator/computer-vision pantry detection
- Fully unrestricted AI-generated recipes
- Voice cooking assistant
- Weekly meal planning
- Nutrition optimization / calorie tracking
- Medical diet recommendations
- Smart kitchen / IoT inventory integration

14. Success Metrics & KPIs

North Star Metric

Completed Cook-at-Home Baskets — the number of Cook-at-Home sessions that successfully result in recipe products being added to the cart.

Funnel

```
Dish Search → Cook at Home Impression → Cook at Home Click → Serving Selected →
Ingredients Generated → Ingredients Reviewed → Add All to Cart → Checkout → Purchase
```

Metric	Definition / Target
Cook-at-Home CTR	Cook at Home Clicks ÷ Card Impressions
Recipe-to-Cart Conversion	Add All to Cart ÷ Recipe Sessions
Recipe Purchase Conversion	Completed Orders ÷ Recipe Sessions
Ingredient Coverage	Matched Ingredients ÷ Required Ingredients (target ≥ 98%)
Critical Ingredient Availability	Target: 100% before showing “Ready to Cook”
SKU Matching Accuracy	Target ≥ 95%
Recipe Generation Latency	Target < 3 seconds (prototype)

Business KPIs to Track

- Average Order Value (AOV) and items per order
- Search-to-cart and checkout conversion
- Repeat cooking sessions and recipe engagement
- Ingredient removal / substitution rate
- Recipe abandonment and availability coverage

Suggested A/B Experiment

Control: standard Blinkit search → product results. Treatment: search → Cook at Home → product results. Compare conversion, basket size, AOV, search abandonment, and checkout rate.

15. Risks & Mitigations

Risk	Mitigation
Incorrect recipe data	Use curated, verified recipes for MVP.
Incorrect ingredient quantities	Deterministic serving-scaling engine + validated recipe data.
Wrong product mapping (e.g. “cream” → face cream)	Ingredient IDs, category restrictions, semantic matching, never keyword-only similarity.
Product out of stock	Real-time inventory validation before basket creation; substitution engine.
Excessive cart price (every spice purchased)	Distinguish essential ingredients vs. pantry staples; highlight savings from pantry removal.
User already owns ingredients	Pantry removal is a first-class step before cart creation, not optional polish.

SWOT Summary

Dimension	Summary
Strengths	Reduces shopping friction; natural quick-commerce fit; increases AOV; improves product discovery.
Weaknesses	Complex SKU mapping; pack sizes rarely equal recipe quantities; recipe accuracy required.
Opportunities	AI meal planning; personalized recipes; pantry intelligence; voice cooking; nutrition features.
Threats	Competitors can replicate; incorrect AI output; low adoption; out-of-stock ingredients; trust loss from wrong substitutions.

16. Competitive Positioning

This should not be framed as inventing recipe technology from nothing — Blinkit has publicly discussed generative-AI recipe work ("Recipe Rover"), which validates the underlying direction rather than undercutting this concept. The differentiation here is the full **recipe discovery → transactional cooking workflow**, specifically: serving-specific quantities, requirement-to-pack transparency, pantry removal, pack optimization, smart substitutions, one-tap complete cart, and a post-delivery cooking mode.

Strategically, quick commerce currently answers "I need something now." Cook at Home extends this to "I want to cook something now," positioning the product to compete not only with other grocery apps but indirectly with food-delivery decisions at the moment a user decides what to eat.

17. Appendix — Core Innovation Pipeline

The project's centerpiece is not an AI recipe generator; it is the end-to-end pipeline connecting user intent to a ready-to-checkout, ready-to-cook basket:

```
USER INTENT ("I want to cook Paneer Butter Masala")
↓
RECIPE ENGINE → SERVING ENGINE → INGREDIENT REQUIREMENTS →
INGREDIENT NORMALIZATION → SKU MATCHING → INVENTORY CHECK →
PACK OPTIMIZER → USER PANTRY → SUBSTITUTION → PRICE CALCULATION →
ONE-TAP BASKET → CHECKOUT → COOKING ASSISTANT
```

Suggested project names: **Blinkit Personal Chef** ("From Recipe Intent to One-Tap Grocery Cart"), **Blinkit Cook at Home**, or — for an AI/ML-focused portfolio — **Recipe2Cart: An AI-Powered Recipe-to-Commerce System for Quick Commerce.**

End of Document — Blinkit Personal Chef PRD v1.0